DA2009 - Group Project - IRCSL Ethical Web Scraping

DA 2009 - Data Collection Methods II

Group 3

Name	Student ID	Phase
Chandrasekaran Sivaneswaran	25ada147	Phase 3 - Brokers and Loss Adjusters
Zainul Abdeen Athif	25ada023	Phase 4 - Document Engine
V. K. Rajapakse	25ada139	Phase 1 - Static Pages
P. P. M. Subasinsani	25ada029	Phase 2 - Insurance Companies

Phase 3 - Brokers and Loss Adjusters: Dynamic Pages, Selenium and APIs

Done by: Chandrasekaran Sivaneswaran (25ada147)
Websites: https://ircsl.gov.lk/insurance-brokers/ and https://ircsl.gov.lk/loss-adjusters/

These directories look dynamic in the browser: paging, a search box and + buttons that open contact details. In this phase we find out whether the data is really loaded by JavaScript. We check the HTML with Requests, automate the browser with **Selenium**, and look for a hidden **API**.

1. Setup

Same rules as Phase 1: robots.txt check, student User-Agent, 3-second delay. We also check the WordPress API address used in Section 5.

In [1]:
```
import os
import json
import time
import urllib.robotparser

import requests
import pandas as pd
from bs4 import BeautifulSoup

USER_AGENT = "DA2009-StudentProject/1.0 (BSc Applied Data Analytics coursework; academic use only)"
DELAY_SECONDS = 3

BASE_URL = "https://ircsl.gov.lk"
BROKERS_URL = BASE_URL + "/insurance-brokers/"
LOSS_ADJUSTERS_URL = BASE_URL + "/loss-adjusters/"
API_URL = BASE_URL + "/wp-json/wp/v2/pages?slug=insurance-brokers"
os.makedirs("data/processed", exist_ok=True)

session = requests.Session()
session.headers.update({"User-Agent": USER_AGENT})

rp = urllib.robotparser.RobotFileParser()
rp.set_url(BASE_URL + "/robots.txt")
rp.read()
for u in [BROKERS_URL, LOSS_ADJUSTERS_URL, API_URL]:
    print("allowed:", rp.can_fetch(USER_AGENT, u), u)


def polite_get(url):
    if not rp.can_fetch(USER_AGENT, url):
        print("Not allowed by robots.txt:", url)
        return None
    try:
        response = session.get(url, timeout=30)
    except requests.exceptions.RequestException as error:
        print("Request failed:", url, error)
```

```
        return None
    time.sleep(DELAY_SECONDS)
    print(response.status_code, url)
    return response if response.status_code == 200 else None
```

```
allowed: True https://ircsl.gov.lk/insurance-brokers/
allowed: True https://ircsl.gov.lk/loss-adjusters/
allowed: True https://ircsl.gov.lk/wp-json/wp/v2/pages?slug=insurance-brokers
```

2. Static or dynamic?

In the browser only 10 rows are shown, with Previous / Next buttons and a search box. We downloaded each page once with Requests. Requests does not run JavaScript, so it only shows what the server sends.

In [2]:
```
brokers_response = polite_get(BROKERS_URL)
loss_response = polite_get(LOSS_ADJUSTERS_URL)
brokers_soup = BeautifulSoup(brokers_response.text, "html.parser")
loss_soup = BeautifulSoup(loss_response.text, "html.parser")

print("\nBroker rows in the HTML:", len(brokers_soup.select("#company-table tbody tr")))
print("Loss adjuster rows in the HTML:", len(loss_soup.select("#loss-adjusters-table tbody tr")))

start = brokers_response.text.find("DataTable({")
print("Table settings in the page source:", " ".join(brokers_response.text[start:start + 60].split()))
```

```
200 https://ircsl.gov.lk/insurance-brokers/
200 https://ircsl.gov.lk/loss-adjusters/

Broker rows in the HTML: 84
Loss adjuster rows in the HTML: 29
Table settings in the page source: DataTable({ "order": [[ 1, "asc" ]] }); // Add
```

All 84 brokers and 29 loss adjusters are already in the HTML, even though the browser shows 10 at a time. The paging, sorting and search come from a JavaScript plugin called **DataTables**, which works on the rows the browser already has.

Each row also has the same hidden JavaScript list of contact details that Phase 2 found on the insurance companies page. Here we read it with BeautifulSoup instead of Scrapy, for both directories at once.

In [3]:
```
def hidden_values(row):
    script = row.find("script")
    if script is None or "new Array(" not in script.get_text():
        return []
    list_text = script.get_text().split("new Array(")[1].split(");")[0]
    return json.loads("[" + list_text.replace("img_to_get", '""') + "]")


def clean(value):
    if not isinstance(value, str):
        return value
    if "&" in value or "<" in value:
        value = BeautifulSoup(value, "html.parser").get_text(" ")
    value = " ".join(value.replace("\xa0", " ").split())
    return None if value in ["", "-", "—"] else value


scraped_at = time.strftime("%Y-%m-%d %H:%M:%S")
broker_rows, loss_rows = [], []
for tr in brokers_soup.select("#company-table tbody tr"):
    cells = [td.get_text(" ", strip=True) for td in tr.find_all("td")]
    h = hidden_values(tr)
    broker_rows.append({"name": cells[1], "class_of_business": cells[2], "registration_period_raw":
cells[3],
                        "principal_officer": h[3], "correspondence_address": h[4], "telephone": h[6],
                        "email": h[8], "website": h[9], "source_url": BROKERS_URL, "scraped_at":
scraped_at})
for tr in loss_soup.select("#loss-adjusters-table tbody tr"):
    cells = [td.get_text(" ", strip=True) for td in tr.find_all("td")]
    h = hidden_values(tr)
    loss_rows.append({"name": cells[1], "licence_date_raw": cells[2], "address": h[1], "telephone":
```

```
h[2],
                        "email": h[3], "source_url": LOSS_ADJUSTERS_URL, "scraped_at": scraped_at})

brokers = pd.DataFrame(broker_rows).apply(lambda col: col.map(clean))
loss_adjusters = pd.DataFrame(loss_rows).apply(lambda col: col.map(clean))
print("Brokers:", len(brokers), "| loss adjusters:", len(loss_adjusters))
brokers.head()
```

```
Brokers: 84 | loss adjusters: 29
```

Out[3]:

	name	class_of_business	registration_period_raw	principal_officer	correspondence_address	telephone	email	website	source_url	scraped_at
0	Bluestone Specialty Insurance Brokers (Pvt) Ltd	General Insurance	30.06.2026 – 29.06.2027	Mr. Samarendra Das	5/5, Bankhill Gardens, Rathnayaka Mawatha, Pel...	0112787536	samarendradas8@gmail.com	NaN	https://ircsl.gov.lk/insurance-brokers/	2026-10-09 11:58:53
1	Swan Insurance Brokers (Pvt) Ltd	Long Term and General Insurance	01.01.2026 – 31.12.2026	Mr. G. A. Saman Indrasiri	No.22, Bodhiraja Mawatha, 7th Lane, Kawdana, D...	0772 908 936	saman.indrasiri@swan.lk	NaN	https://ircsl.gov.lk/insurance-brokers/	2026-10-09 11:58:53
2	Cover and Care Insurance Brokers (Pvt) Ltd	Long Term and General Insurance	03.11.2025 - 02.11.2026	Mr. K W M Ananda Kulathunga	No.246, 2nd Floor, Janajaya City Mall, Rajagiriya	0778155156, 0773865489	hello@coverandcare.lk	NaN	https://ircsl.gov.lk/insurance-brokers/	2026-10-09 11:58:53
3	Phoenix Insurance Brokers and Consultants (Pvt...	Long Term and General Insurance	22.04.2026 - 31.12.2026	Mr Gihan Adikari	No.367/19, Lurdhu Mw, Galmankada, Kimbulapitiy...	0777627527 / 0777565729	ins.phoenixins@outlook.com / inf.pibc@outlook.com	NaN	https://ircsl.gov.lk/insurance-brokers/	2026-10-09 11:58:53
4	Ceylan Insurance & Re-insurance Brokers Compan...	General Insurance	01.01.2026 – 31.12.2026	Mr. Priyanga M. N. De Silva	No. 1, 3/1, 3rd Floor, Deanston Place, Colombo 03	+94114114114 / +94770127000	principalofficer@ceylaninsurance.com	www.ceylaninsurance.com	https://ircsl.gov.lk/insurance-brokers/	2026-10-09 11:58:53

2.1 Licence status

Broker registration periods look like `30.06.2026 – 29.06.2027`, written with different dashes and spaces. We remove the spaces, change the long dash into a hyphen, split the start and end dates, and compare them with today's date. Three loss adjusters say `Re-registered on ...`, so we keep that as its own column.

In [4]:

```
TODAY = pd.Timestamp.today().normalize()
periods = brokers["registration_period_raw"].str.replace(" ", "").str.replace("–", "-").str.split("-")
brokers["registration_start"] = pd.to_datetime(periods.str[0], format="%d.%m.%Y", errors="coerce")
brokers["registration_end"] = pd.to_datetime(periods.str[-1], format="%d.%m.%Y", errors="coerce")
brokers["licence_status"] = "Active"
brokers.loc[brokers["registration_end"] < TODAY, "licence_status"] = "Expired"
brokers.loc[brokers["registration_start"] > TODAY, "licence_status"] = "Not yet started"

loss_adjusters["re_registered"] = loss_adjusters["licence_date_raw"].str.contains("Re-registered")
date_text = loss_adjusters["licence_date_raw"].str.replace("Re-registered on", "").str.strip()
loss_adjusters["licence_effective_date"] = pd.to_datetime(date_text, format="%d.%m.%Y")

print("Date used:", TODAY.date())
print("Broker licence status:", brokers["licence_status"].value_counts().to_dict())
print("Loss adjusters re-registered:", loss_adjusters["re_registered"].sum())
```

```
Date used: 2026-10-09
Broker licence status: {'Active': 84}
Loss adjusters re-registered: 3
```

3. Selenium: automating the browser

Requests only sees the page before JavaScript runs. Selenium controls a real browser from Python, so we can test the dynamic parts. Selenium gets the matching ChromeDriver for Chrome by itself.

In [5]:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
```

3.1 Opening the page in a headless browser

Headless means Chrome runs without opening a window. Selenium does not check robots.txt by itself, so we check it first. We use `WebDriverWait` to wait until the first name in the table is visible, because DataTables needs a moment to draw the table.

In [6]:
```
options = webdriver.ChromeOptions()
for arg in ["--headless=new", "--no-sandbox", "--disable-dev-shm-usage", "--window-size=1400,1000",
            "--user-agent=" + USER_AGENT]:
    options.add_argument(arg)

driver = webdriver.Chrome(options=options)
wait = WebDriverWait(driver, 20)

if rp.can_fetch(USER_AGENT, BROKERS_URL):
    driver.get(BROKERS_URL)
    wait.until(EC.visibility_of_element_located((By.CSS_SELECTOR, "#company-table tbody tr td:nth-
child(2)")))

print("Rows shown in the browser:", len(driver.find_elements(By.CSS_SELECTOR, "#company-table tbody
tr")))
print("Table info text:", driver.find_element(By.ID, "company-table_info").text)
```

```
Rows shown in the browser: 10
Table info text: Showing 1 to 10 of 84 entries
```

3.2 Clicking Next through all the pages

We click Next until the button is disabled. After each click JavaScript removes the old rows and adds new ones, so we wait with `EC.staleness_of()` until the old first row has gone. This is better than `time.sleep()` because it only waits as long as needed. We read each name with `get_attribute("textContent")`.

We also wanted to see whether paging downloads any data. The browser keeps a list of every file it downloads (the same list as the Network tab in DevTools), which we read with `driver.execute_script()` before and after paging.

In [7]:
```
def downloaded_urls():
    return driver.execute_script("return performance.getEntriesByType('resource').map(e => e.name)")


urls_before = downloaded_urls()
selenium_names = []
pages_read = 0
clicks = 0
start = time.time()

while True:
    rows = driver.find_elements(By.CSS_SELECTOR, "#company-table tbody tr")
    for r in rows:
        name_cell = r.find_element(By.CSS_SELECTOR, "td:nth-child(2)")
        selenium_names.append(name_cell.get_attribute("textContent").strip())
    pages_read += 1

    next_button = driver.find_element(By.ID, "company-table_next")
    if "disabled" in next_button.get_attribute("class"):
        break

    next_button.click()
    clicks += 1
    wait.until(EC.staleness_of(rows[0]))

print("Pages read:", pages_read, "| clicks:", clicks, "| brokers collected:", len(selenium_names),
      "| time:", round(time.time() - start, 1), "seconds")
```

```
Pages read: 9 | clicks: 8 | brokers collected: 84 | time: 0.7 seconds
```

Now we compare the results. The browser sorts the table by name, so we compare the two lists as sets. For the new downloads, we only count a file as possible data if it comes from ircsl.gov.lk and is not an image, style or script file.

In [8]:
```
new_urls = downloaded_urls()[len(urls_before):]
data_downloads = [u for u in new_urls if "ircsl.gov.lk" in u.split("/")[2]
                  and not u.split("?")[0].lower().endswith((".png", ".jpg", ".gif", ".svg", ".ico",
".css", ".js"))]

print("New downloads while paging:", len(new_urls), "| downloads that could carry data:",
len(data_downloads))
print("Same brokers as Requests:", set(brokers["name"]) == set(clean(n) for n in selenium_names))
```

```
New downloads while paging: 0 | downloads that could carry data: 0
Same brokers as Requests: True
```

While paging the browser downloaded **no data**, only small icons and a Google Analytics request, and it found the same 84 brokers. All the rows were already in the browser; JavaScript just showed a different 10 each time.

3.3 Clicking + to open the details box

When the first cell of a row is clicked, JavaScript adds a new row under it with the contact details. We wait for this row to appear (`tr.shown + tr`) and read it. Selenium's `.text` gives the text as shown on screen, and the website's CSS shows the title in capitals, so we used `textContent` .

In [9]:
```
driver.find_element(By.CSS_SELECTOR, "#company-table_paginate a[data-dt-idx='1']").click()
wait.until(EC.text_to_be_present_in_element((By.ID, "company-table_info"), "Showing 1 to"))

driver.find_element(By.CSS_SELECTOR, "#company-table tbody tr td.details-control").click()
panel = wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "#company-table tbody tr.shown +
tr")))

title = panel.find_element(By.CSS_SELECTOR, ".title")
print("Using .text:", title.text, "| using textContent:", title.get_attribute("textContent"))

email_box = panel.find_element(By.CSS_SELECTOR, ".email")
label = email_box.find_element(By.TAG_NAME, "b").get_attribute("textContent")     # "Email :"
panel_email = email_box.get_attribute("textContent").replace(label, "")
same = brokers[brokers["name"] == clean(title.get_attribute("textContent"))].iloc[0]
print("Email after clicking:", clean(panel_email), "| email from the hidden list:", same["email"])
```

```
Using .text: 3 D H INSURANCE BROKERS (PVT) LTD. | using textContent: 3 D H Insurance Brokers (Pvt) Ltd.
Email after clicking: info@3dhins.lk, lal@3dhins.lk | email from the hidden list: info@3dhins.lk,
lal@3dhins.lk
```

4. Looking for a hidden API

Some websites load their table data from an API (an XHR or fetch request that returns JSON). If there was one, using it would be better than reading the HTML or using a browser. We checked two things:

1. the browser's list of XHR / fetch requests (like DevTools -> Network -> Fetch/XHR)
2. the public WordPress REST API at `/wp-json/` , read with `requests` and `.json()`

In [10]:
```
entries = driver.execute_script(
    "return performance.getEntriesByType('resource').map(e => ({type: e.initiatorType, url:
e.name}));")
driver.quit()

network = pd.DataFrame(entries)
xhr = network[network["type"].isin(["xmlhttprequest", "fetch"])]
print("XHR / fetch requests:", xhr["url"].str[:80].tolist())
```

```
api_response = polite_get(API_URL)
api_page = api_response.json()[0]
api_content = api_page["content"]["rendered"]
print("\nAPI content type:", api_response.headers["Content-Type"])
print("API page title:", api_page["title"]["rendered"])
print("API content contains a broker name:", brokers["name"].iloc[0] in api_content)
```

```
XHR / fetch requests: ['https://www.google-analytics.com/g/collect?v=2&tid=G-
JQFYBD5YVM&gtm=45Pe6a72v920']
200 https://ircsl.gov.lk/wp-json/wp/v2/pages?slug=insurance-brokers

API content type: application/json; charset=UTF-8
API page title: Insurance Broking Companies
API content contains a broker name: False
```

The only fetch request goes to Google Analytics, and the WordPress API only gives the introduction text of the page; the table is added by the website template. So there is no hidden API with the broker data.

Method	Brokers found	Contact details	Result
Requests + BeautifulSoup	84	Yes (hidden list)	Best: complete, fast, 1 request
Selenium	84	Only by clicking + on every row	Works, but slower; only needed if JavaScript loaded the data
WordPress REST API	0	No	No broker data

5. Saving the results

Some brokers use personal email addresses (Gmail, Yahoo, Hotmail). We mark them in a `personal_email` column, which we discuss in the ethics section.

In [11]:
```
PERSONAL = ["gmail.com", "yahoo.com", "hotmail.com", "outlook.com"]
for df in [brokers, loss_adjusters]:
    df["personal_email"] = df["email"].fillna("").str.lower().apply(lambda e: any(p in e for p in
PERSONAL))

brokers.to_csv("data/processed/insurance_brokers.csv", index=False)
brokers.to_json("data/processed/insurance_brokers.json", orient="records", indent=2,
date_format="iso")
loss_adjusters.to_csv("data/processed/loss_adjusters.csv", index=False)
loss_adjusters.to_json("data/processed/loss_adjusters.json", orient="records", indent=2,
date_format="iso")

print("Brokers:", len(brokers), "| active:", (brokers["licence_status"] == "Active").sum(),
      "| personal emails:", brokers["personal_email"].sum())
print("Loss adjusters:", len(loss_adjusters), "| re-registered:",
loss_adjusters["re_registered"].sum(),
      "| personal emails:", loss_adjusters["personal_email"].sum())
```

```
Brokers: 84 | active: 84 | personal emails: 17
Loss adjusters: 29 | re-registered: 3 | personal emails: 25
```

6. Ethical considerations and limitations

- We checked robots.txt for every URL, including before Selenium opened the page, because Selenium does not check it by itself.
- We waited 3 seconds after every request, and Selenium opened the page only once. Both Requests and the browser used our student User-Agent.
- Requests + BeautifulSoup was our main method. We used Selenium only to test the dynamic parts, and did not click + on all 84 rows because the data was already in the HTML.
- The officer names and contact details are official information published by IRCSL. Some brokers use personal email addresses, which we marked. We only use this data for the assignment.

- **Limitations:** the licence status depends on the date the notebook is run, and loss adjusters only have a start date, so we could not work out an expiry date.